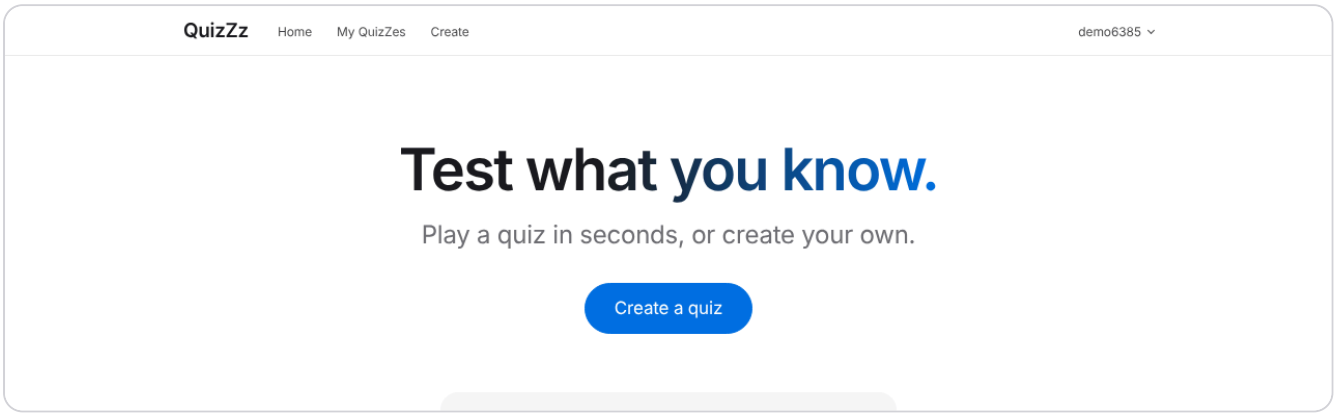
QuizZz.
An online quiz system.

A full-stack Java web application built on Spring and Spring Boot: how it was found, how it works, what it is made of, how it was run and redesigned, and what we verified along the way.



3,694	63	14	42
lines of Java	unit tests passing	page templates	screenshots

Summary

The goal was to build a Java **online quiz system with Spring and Spring Boot**. Rather than start from nothing, we searched public GitHub for the closest existing projects, ranked them, and chose the leader: **oojorgeoo89/QuizZz**, the most-starred Spring Boot quiz application we could find. We cloned it into this repository, got it running in a clean cloud environment, redesigned its entire interface in a minimal, Apple-inspired style, photographed every working screen, and documented the system in this report.

Users can register by email, build quizzes made of questions and answers, publish them, and let anyone play and be scored instantly. Behind that sit a layered Spring architecture, token-based email flows, aspect-oriented access control and a REST API.

Inside this report

1 · Overview
How the base project was found and what it does

2 · Architecture
Layers, data model, code organisation

3 · Technology stack
Every component, and why Spring Boot

4 · Security and running it
Safeguards, setup steps, problems solved

5 · User interface
Critique and redesign, dark mode, mobile

6 · Verification
Tests, limitations, next steps

What we set out to do, and how we chose the base

Start from the strongest open-source example instead of a blank project, then make it ours.

The search

We searched public GitHub repositories for Java projects matching “online quiz system”, “quiz application” and “online examination system” combined with Spring Boot, and ranked the results by stars. Most matches are small student projects with zero to six stars; one stands far above the rest.

RANK	REPOSITORY	STARS	NOTES
1	oojorgeoo89/QuizZz	86	66 forks, MIT licence. Spring MVC, Security, Data JPA, AOP, Thymeleaf, AngularJS
2	Philmasteryeah/quizmaker	9	Spring Boot 2, Thymeleaf, AdminLTE, user management
3	krishnalagad/classroom-quiz-...	6	MCQ exams with Angular front end, JWT
4	dumbestprogrammer/... QuizApplication	5	Quiz generation, REST API
4	ayushcode12/quiz-microservices-app	5	Microservices with Eureka and API gateway

QuizZz was chosen for both popularity and fit: it uses the exact stack requested (Spring and Spring Boot) and is deliberately a *learning project* that exercises many parts of the Spring family, which makes it a good foundation and a good thing to document.

What the application does

For players

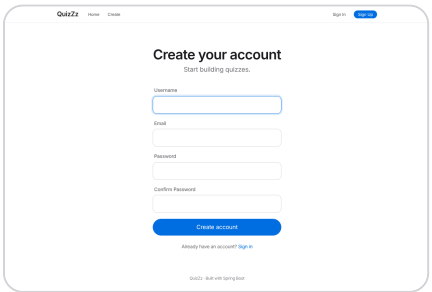
- Browse published quizzes on the home page
- Play without needing an account
- Answer one question at a time
- See an instant percentage score

For authors (registered users)

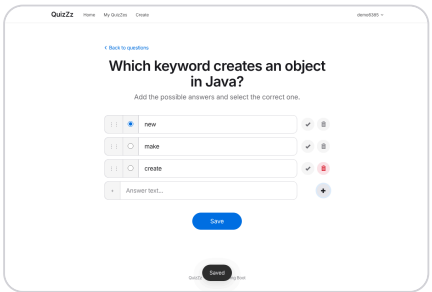
- Sign up with email activation
- Create quizzes, add and reorder questions by drag and drop
- Add answers and mark the correct one
- Save as draft, then publish once well-formed

Account recovery is built in: a “forgot password” flow emails a one-time link that opens a reset form. Only the owner of a quiz can edit or delete it.

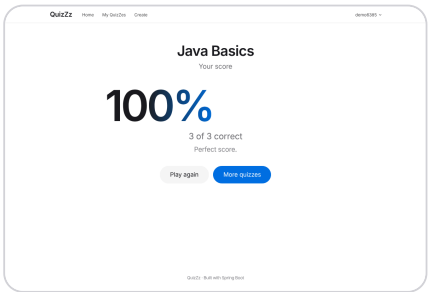
The user journey we exercised end to end



Sign up



Build a quiz



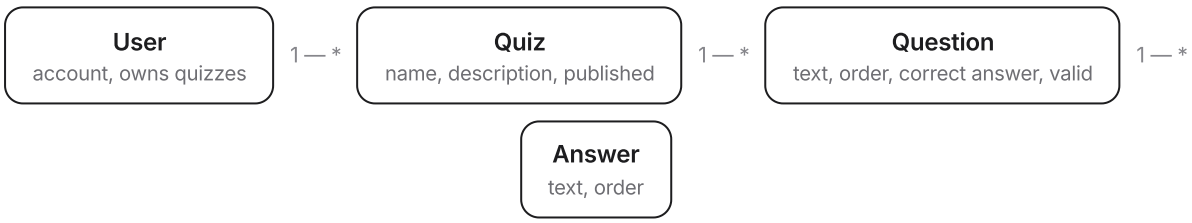
Play and score

A classic layered Spring application

Each layer has one job and only talks to the layer beneath it, so behaviour can be tested and changed in isolation.

Browser	Server-rendered pages (Thymeleaf) plus AngularJS apps that call the REST API for lists, the quiz editor and the quiz player
▼	
Controllers	Web controllers return pages; REST controllers (`/api/…`) return JSON. Exception handlers turn errors into friendly pages or JSON
▼	
Services	Business rules for quizzes, questions, answers, users, tokens and registration. Wrapped by AOP aspects that enforce who may do what
▼	
Repositories	Spring Data JPA interfaces; no hand-written SQL for normal CRUD, built-in paging and sorting
▼	
Database	H2 (embedded, zero setup) or MySQL, chosen with a Spring profile; Hibernate maps entities to tables

Data model



Supporting tables: `RegistrationToken` and `ForgotPasswordToken` (one-time email links with an expiry) and `persistent_logins` (remember-me sessions). A question is only “valid”, and a quiz only publishable, when it has answers and a correct answer.

Code organisation (`jorge.rv.quizzz`)

PACKAGE	RESPONSIBILITY
config	Security, MVC, mail, async, scheduling and AOP configuration
controller.web / .rest.v1	Page controllers and the versioned JSON API for quizzes, questions, answers and users
service	Business logic; `accesscontrol` holds ownership rules and their aspects; `usermanagement` holds registration, reset and token delivery
repository · model	Spring Data repositories and JPA entities
tasks · exceptions	Scheduled token clean-up; a typed exception hierarchy

Example: playing a quiz

1. The page `/quiz/{id}/play` loads and the Angular player requests `GET /api/quizzes/{id}/questions?onlyValid=true` .
2. For each question it fetches the answers (`/api/questions/{id}/answers`) and shows them as choices.
3. After the last question it posts all selections to `/api/quizzes/{id}/submitAnswers` ; the service compares them to the correct answers and returns the score.

What it is built with, and why

TECHNOLOGY	ROLE	WHY WE USE IT
Java 8	Language / runtime	The version this Spring Boot generation targets; stable and universally supported
Spring Boot 1.5	Application framework	Auto-configuration, "starter" dependencies, embedded Tomcat and one-command start; replaces pages of XML configuration
Spring MVC	Web and REST layer	One programming model for both HTML pages and JSON endpoints; Jackson handles JSON
Thymeleaf	Page templates	Natural HTML templates, built-in i18n through `messages.properties`, and `sec:` tags that show or hide UI by login state
Spring Data JPA + Hibernate	Persistence	Declare an interface, get CRUD, paging and sorting; Hibernate maps entities and can create the schema
H2 / MySQL	Database	H2 runs in-process for development and demos; MySQL is the production default. A profile switch selects either
Spring Security	Authentication	Form login, remember-me stored in the database, BCrypt password hashing, HTTP Basic for the API
Spring AOP / AspectJ	Access control	Ownership checks wrap repository calls once, rather than being repeated in every service method
Spring Mail + @Async	Email	Sends activation and reset links without making the user wait for the mail server
Spring Scheduling	Housekeeping	A task runs every 20 seconds and removes expired registration and reset tokens
AngularJS 1.3	Interactive front end	Data-bound lists, editor and player talking to the REST API; drag-and-drop ordering via angular-drag-and-drop-lists
Bootstrap 3 + jQuery	UI base	Grid, form and dropdown behaviour; bundled as webjars so no CDN is needed. Our design layer restyles it
Maven	Build	Dependency management and a repeatable build into one runnable WAR
JUnit, Mockito, GreenMail	Testing	Fast unit tests for services; GreenMail provides a fake SMTP server for integration tests

Why Spring Boot specifically

Less plumbing

Starters bring a compatible set of libraries in one line, and auto-configuration wires them from what is on the classpath. We write business code, not setup.

Runs anywhere

The app is an executable archive with its own server inside: `java -jar` and it is live. No separate application server to install.

Profiles

The same build runs against MySQL or H2 by changing one property, which is exactly how we ran it in a container with no database installed.

Batteries included

Security, mail, scheduling, validation, error handling and testing support all come from one consistent ecosystem.

How it stays safe, and how we got it running

Security and reliability design

- **Two security chains.** `/api/**` uses HTTP Basic; the rest of the site uses a login form with remember-me persisted in the database. Passwords are hashed with BCrypt.
- **Ownership by aspect.** `@Around` advice on the repositories checks that the signed-in user may read, save or delete an object, so a user cannot edit another user's quiz even through the API.
- **One-time email tokens.** Registration and password reset use random UUID tokens with an expiry; a scheduled task purges expired ones.
- **Non-blocking mail.** Delivery is `@Async`, and a console delivery implementation exists for development.
- **Typed errors.** A custom exception hierarchy is mapped to friendly error pages or JSON error bodies.

How we built and ran it

1. **Cloned** QuizZz into this repository and removed the earlier planning documents.
2. **Matched the runtime.** The container shipped Java 21, but Spring Boot 1.5 targets Java 8, so we installed OpenJDK 8 and built with Maven.
3. **Chose the database.** With no MySQL available we started with the `h2` profile, a file-backed embedded database.
4. **Replaced external CDNs.** Bootstrap and jQuery were loaded from public CDNs that are unreachable in the sandbox, leaving pages unstyled. We pointed the templates at the copies already bundled as webjars.
5. **Caught the emails.** Sign-up and password reset need SMTP, so we ran a tiny local mail sink and a browser-automation script that reads the activation and reset links, letting us exercise every flow.

```
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
mvn -DskipTests package
$JAVA_HOME/bin/java -jar target/QuizZz-0.0.1-SNAPSHOT.war \
    --spring.profiles.active=h2          # then open http://localhost:8080
```

Problems met, and fixes

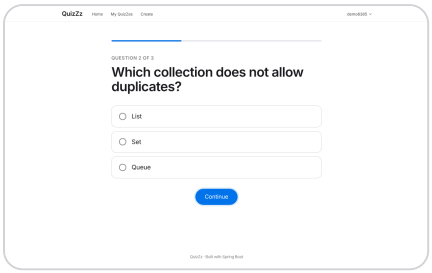
PROBLEM	CAUSE	FIX
Wrong Java version	Container default was Java 21; Spring Boot 1.5 targets Java 8	Install and use OpenJDK 8
Pages unstyled	CDN hosts blocked in the sandbox	Serve Bootstrap and jQuery from webjars
Sign-up could not complete	No SMTP server on port 25	Local mail sink to capture the links
Placeholder numbers on lists	"20" and "5033" hardcoded in two templates	Removed from the redesigned UI

REST API at a glance

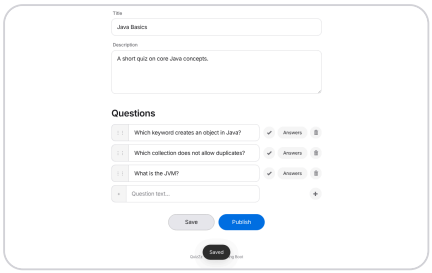
RESOURCE	OPERATIONS
/api/quizzes	List published quizzes (paged, sorted); get, create, delete a quiz; `POST /{id}/publish`; `GET /{id}/questions`; `POST /{id}/submitAnswers`; `GET /search`
/api/questions	Get, create, update, delete a question; `GET /{id}/answers`; `GET|POST /{id}/correctAnswer`
/api/answers	Get, create, update, delete an answer
/api/users	Registration and password-reset steps; `GET /{id}/quizzes` for a user's own quizzes

Redesigned to be minimal and sleek

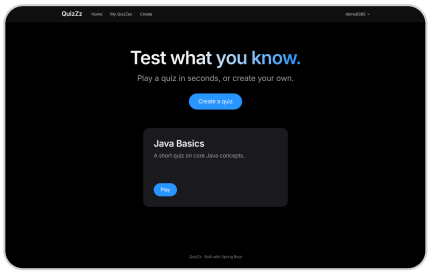
The original used stock Bootstrap. We kept every working behaviour and replaced the look with a restrained, Apple-inspired system.



Quiz player with progress



Editor with “Saved” toast



Dark mode

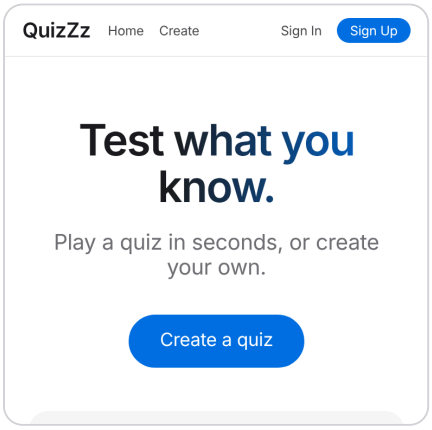
Design principles

- **Quiet and typographic.** The system font stack (San Francisco on Apple devices), large tight headings, generous whitespace, one accent colour (`#0071E3`).
- **Soft surfaces.** Light-grey rounded cards instead of tables and borders; pill buttons; a translucent navigation bar that stays in view.
- **Adaptive.** Colours are CSS variables, so dark mode follows the device setting; layouts reflow to phone width.

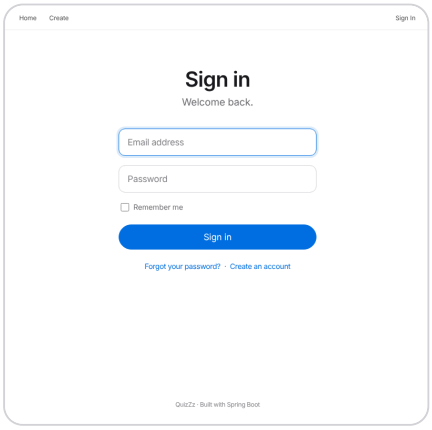
Critique and what we changed

WHAT WAS WRONG	WHAT WE DID
Heavy tables with fake figures	Card grid; placeholder counts removed; empty states added
Browser `alert()` pop-ups	Non-blocking toasts, plus “Saved / Deleted / Published” confirmations
No sense of progress while playing	Progress bar, “Question 2 of 3”, and a result message
Ambiguous pencil icon	Labelled “Answers” button; back links in editors
Only a tiny button was clickable	Whole quiz card is the Play target
No footer, favicon, mobile viewport or dark mode	All added; entrance motion respects reduced-motion settings

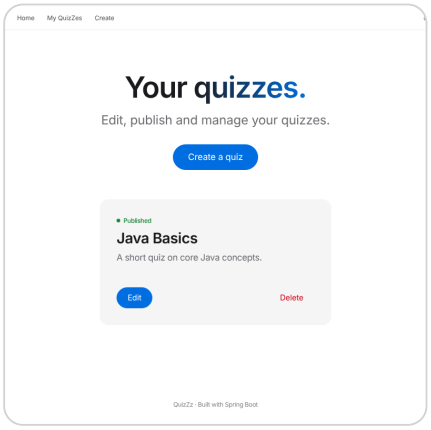
We reviewed screenshots after each pass and fixed what we saw: duplicate toasts were merged, headline gradients now shrink-wrap their text so the colour is visible, and a lone quiz card is centred rather than stranded on one side.



Phone width



Sign in



My quizzes

What we verified, what is left

Verification

- **Unit tests:** 63 tests across the quiz, question, answer and user-management services ran and passed with Maven.
- **End-to-end walkthrough:** a scripted browser session registered a user, confirmed the email link, logged in, created a quiz with three questions and nine answers, published it, played it to 100%, logged out, reset the password from the emailed link and logged in again.
- **Screenshots:** 42 images in the `screenshots/` folder cover every page and state: 33 desktop screens, 4 in dark mode and 5 at phone width.

What we did not run. The integration test (`LifeCycleTest`) targets a MySQL test database, which was not available here, so only the unit tests were executed. We exercised the same life-cycle through the browser instead.

Known limitations

- **Aging platform.** Spring Boot 1.5 (2017), AngularJS 1.3 and Bootstrap 3 are all past open-source support. They work, but should not face the public internet unchanged.
- **CSRF protection is disabled** in the security configuration. Acceptable for a demo, but it should be enabled for production.
- **No search box.** A search endpoint exists in the API but the interface does not expose it.
- **Feedback gaps.** Answers are not revealed per question, only the final score.
- **Local data.** The H2 file database and the mail sink live in the sandbox and disappear with it.

Suggested next steps

PRIORITY	STEP	BENEFIT
High	Upgrade to current Spring Boot and Java 17+; enable CSRF	Security fixes, modern tooling
High	Add automated browser tests from our walkthrough script	Protects the redesigned UI
Medium	Search UI, categories, per-question feedback, timers	Richer quiz experience
Medium	Replace AngularJS with a maintained front end	Long-term maintainability
Low	Containerise with Docker and MySQL; CI pipeline	One-command deployment

Repository

Everything lives in `Yash-912/PORP` on branch `claude/online-quiz-github-research-yjasc5` :

```
src/main/java        controllers, services, repositories, models
src/main/resources   templates/ · static/css/main.css · static/js/ · messages.properties
screenshots/         42 images of the running application
report.pdf           this document
```